

Request Handling

Section: 8 — Request/Response Handling, Forms, and Validation

Subtopic: 01 — Request Handling

Estimated Duration: 2.5–3 hours

Theory

Practical

Topics covered: `req.params` optional parameters and wildcards, `req.query` arrays and nested objects, `req.body` size limits, file uploads with `multer`, `req.headers`, combining multiple input sources

Prerequisites: Subtopic 07-01 (`req.query` basics), 07-02 (`req.params` fundamentals), 07-03 (`express.json`, `express.urlencoded`)

Learning Objectives — This Subtopic

- Use optional route parameters and wildcards to build flexible URL patterns with `req.params`
- Parse arrays and nested objects from query strings with `req.query`
- Configure body parser size limits to protect against oversized payloads
- Read and use incoming HTTP headers via `req.headers` and `req.get()`
- Handle file uploads with the `multer` middleware and access uploaded file metadata
- Combine data from params, query, body, headers, and files in a single route handler

Where Request Data Lives

When a client sends an HTTP request to an Express server, the data it sends can arrive in several different places. Understanding where each type of data lives on the `req` object is the foundation of request handling.

Data Source	Property	Example URL / Request	Typical Use
URL path segments	<code>req.params</code>	<code>/users/42</code>	Identifying a specific resource
Query string	<code>req.query</code>	<code>/search?term=node&page=2</code>	Filtering, sorting, pagination
Request body	<code>req.body</code>	POST/PUT payload	Creating or updating resources
HTTP headers	<code>req.headers</code>	Authorization: Bearer ...	Authentication tokens, content type
Uploaded files	<code>req.file</code> / <code>req.files</code>	Multipart form data	Images, documents, CSV imports

The following sections cover each source in detail with runnable examples. Set up a project to follow along:

```
mkdir request-handling && cd request-handling
```

```
npm init -y
```

```
npm install express
```

```
request-handling/  
├─ node_modules/  
├─ package.json  
├─ package-lock.json  
└─ app.js
```

req.params — Route Parameters (Continued)

Cross-reference: The fundamentals of `req.params` — single parameters, multiple parameters, the fact that values are always strings, and `parseInt` conversion — were covered in **subtopic 07-02 (Routing), Section 2**. This section covers additional patterns not introduced there.

Optional Parameters and Wildcards

Express supports optional parameters with a `?` suffix:

app.js

```
const express = require('express');
const app = express();

app.get('/articles/:year/:month?', function (req, res) {
  console.log('params:', req.params);
  res.json({
    year: req.params.year,
    month: req.params.month || 'all'
  });
});

app.listen(3000, function () {
  console.log('Server running on http://localhost:3000');
});
```

```
curl http://localhost:3000/articles/2025/06
```

Output:

```
{"year": "2025", "month": "06"}
```

```
curl http://localhost:3000/articles/2025
```

Output:

```
{"year": "2025", "month": "all"}
```

When `:month?` is not present in the URL, `req.params.month` is `undefined`. The code uses the logical OR operator to fall back to `"all"`.

`req.query` — Query String Parameters (Continued)

Cross-reference: The basics of `req.query` — automatic parsing, accessing values, and the fact that values are always strings — were introduced in **subtopic 07-01 (Express Introduction), Section 7** and explored further in **subtopic 07-02 (Routing), Section 3** with filtering and sorting examples. This section covers additional patterns.

Arrays and Nested Objects in Query Strings

Express can parse arrays and nested structures from query strings:

`app.js`

```
const express = require('express');
const app = express();

app.get('/filter', function (req, res) {
  console.log('query:', req.query);
  res.json(req.query);
});

app.listen(3000, function () {
  console.log('Server running on http://localhost:3000');
});
```

```
curl "http://localhost:3000/filter?colour=red&colour=blue&sort[field]=name&sort[order]=asc"
```

Output:

```
// Terminal
query: { colour: [ 'red', 'blue' ], sort: { field: 'name', order: 'asc' } }

// Response
{"colour":["red","blue"],"sort":{"field":"name","order":"asc"}}
```

Repeating the same key (`colour=red&colour=blue`) produces an array. Bracket notation (`sort[field]=name`) produces a nested object. This behaviour is powered by the `qs` library which Express uses internally by default.

Warning: Never trust query string values blindly. A client can send any string they like. Always validate and sanitise query parameters before using them in database queries, file paths, or any operation that could be exploited. Validation is covered in detail in subtopic 08-03.

Cross-reference: The comparison between `req.params` and `req.query` — when to use each, with a reference table and practical examples combining both — was covered in **subtopic 07-02 (Routing), Section 3**.

req.body — Request Body (Continued)

Cross-reference: The body-parsing middleware `express.json()` and `express.urlencoded()` — what they do, how to register them, the `{ extended: true }` option, and what happens when they are missing (`req.body` is `undefined`) — were covered in **subtopic 07-03 (Middleware), Built-in Middleware section**. This section covers additional configuration options.

Body Size Limits

Both body parsers accept a `limit` option that restricts the maximum body size. The default is `100kb`. If a client sends a payload larger than the limit, Express responds with a `413 Payload Too Large` error:

app.js (with custom limit)

```
const express = require('express');
const app = express();

// Allow JSON bodies up to 1 MB
app.use(express.json({ limit: '1mb' }));

// Allow form data up to 500 KB
app.use(express.urlencoded({ extended: true, limit: '500kb' }));

app.post('/api/data', function (req, res) {
  res.json({ received: true, size: JSON.stringify(req.body).length });
});

app.listen(3000, function () {
  console.log('Server running on http://localhost:3000');
});
```

Output:

```
// Sending a body within the limit:
{"received":true,"size":42}

// Sending a body that exceeds the limit:
PayloadTooLargeError: request entity too large
```

Setting appropriate size limits protects your server from receiving excessively large payloads that could exhaust memory.

req.headers — HTTP Headers

Every HTTP request carries headers — metadata about the request such as content type, accepted response formats, authentication tokens, and custom values. Express provides all headers as a plain object via `req.headers`, with header names normalised to **lowercase**.

app.js

```
const express = require('express');
const app = express();

app.get('/headers', function (req, res) {
  console.log('All headers:', req.headers);
  console.log('User-Agent:', req.headers['user-agent']);
  console.log('Accept:', req.headers['accept']);

  res.json({
    userAgent: req.headers['user-agent'],
    accept: req.headers['accept'],
    customHeader: req.headers['x-custom-id'] || 'not provided'
  });
});

app.listen(3000, function () {
  console.log('Server running on http://localhost:3000');
});
```

```
curl http://localhost:3000/headers -H "X-Custom-ID: req-7891"
```

Output:

```
// Terminal (partial output)
User-Agent: curl/8.4.0
Accept: */*

// Response
{"userAgent":"curl/8.4.0","accept":"*/*","customHeader":"req-7891"}
```

Notice that the custom header was sent as `X-Custom-ID` but is accessed as `req.headers['x-custom-id']` in lowercase. Express also provides a convenience method `req.get('X-Custom-ID')` which performs a case-insensitive lookup:

app.js (using req.get)

```
app.get('/auth-check', function (req, res) {
  const token = req.get('Authorization');
  console.log('Authorization header:', token);

  if (!token) {
    return res.status(401).json({ error: 'No authorization header' });
  }

  res.json({ token: token });
});
```

```
curl http://localhost:3000/auth-check -H "Authorization: Bearer eyJhbGciOiJIUzI1NiJ9"
```

Output:

```
{"token": "Bearer eyJhbGciOiJIUzI1NiJ9"}
```

Note: `req.get()` and `req.header()` are aliases — both do the same thing. They are preferred over direct `req.headers[]` access when you want case-insensitive header lookups without worrying about normalisation.

File Uploads with `multer`

Neither `express.json()` nor `express.urlencoded()` can handle file uploads. Files are sent using the `multipart/form-data` content type, which requires a specialised parser. **Multer** is the standard middleware for handling multipart form data in Express.

```
npm install multer
```

Stage 1 Single File Upload

Start with the simplest case — accepting a single file upload and saving it to disk.


```
request-handling/
├─ node_modules/
├─ uploads/           ← multer will save files here
├─ package.json
├─ package-lock.json
└─ app.js
```

```
mkdir uploads  app.js
```

```
const express = require('express');
const multer = require('multer');
const path = require('path');
const app = express();

// Configure multer to save files to the "uploads" directory
const upload = multer({ dest: path.join(__dirname, 'uploads') });

// "avatar" is the field name the client uses in the form
app.post('/upload', upload.single('avatar'), function (req, res) {
  console.log('File info:', req.file);
  res.json({
    message: 'File uploaded',
    filename: req.file.filename,
    originalName: req.file.originalname,
    size: req.file.size,
    mimetype: req.file.mimetype
  });
});

app.listen(3000, function () {
  console.log('Server running on http://localhost:3000');
});
```

Test the upload using `curl` with the `-F` flag, which sends multipart form data. Create a small test file first:

```
echo "Hello, this is a test file." > testfile.txt
```

```
curl -X POST http://localhost:3000/upload -F "[email protected]"
```

Output:

```
// Terminal
File info: {
  fieldname: 'avatar',
  originalname: 'testfile.txt',
  encoding: '7bit',
  mimetype: 'text/plain',
  destination: '/path/to/request-handling/uploads',
  filename: 'a1b2c3d4e5f6a7b8c9d0e1f2',
  path: '/path/to/request-handling/uploads/a1b2c3d4e5f6a7b8c9d0e1f2',
  size: 28
}

// Response
{
  "message": "File uploaded",
  "filename": "a1b2c3d4e5f6a7b8c9d0e1f2",
  "originalName": "testfile.txt",
  "size": 28,
  "mimetype": "text/plain"
}
```

Multer saves the file with a random hex name (no extension) in the `uploads` directory. The `req.file` object contains metadata: the original name, MIME type, file size in bytes, and the path where multer saved the file. The field name `'avatar'` in `upload.single('avatar')` must exactly match the form field name the client uses.

Stage 2 Custom Storage with Original Filenames

The default `dest` option strips the file extension. For more control, use `multer.diskStorage()` to configure the destination directory and filename format:

app.js

```
const express = require('express');
const multer = require('multer');
const path = require('path');
const app = express();

const storage = multer.diskStorage({
  destination: function (req, file, cb) {
    cb(null, path.join(__dirname, 'uploads'));
  },
  filename: function (req, file, cb) {
    // Generate a unique name: timestamp + original extension
    const uniqueName = Date.now() + '-' + Math.round(Math.random() * 1000);
    const ext = path.extname(file.originalname);
  }
});
```

```
        cb(null, uniqueName + ext);
    }
});

const upload = multer({ storage: storage });

app.post('/upload', upload.single('avatar'), function (req, res) {
    console.log('Saved as:', req.file.filename);
    res.json({
        message: 'File uploaded',
        filename: req.file.filename,
        path: req.file.path
    });
});

app.listen(3000, function () {
    console.log('Server running on http://localhost:3000');
});
```

```
curl -X POST http://localhost:3000/upload -F "[email protected]"
```

Output:

```
// Terminal
Saved as: 1750070412345-284.txt

// Response
{
  "message": "File uploaded",
  "filename": "1750070412345-284.txt",
  "path": "/path/to/request-handling/uploads/1750070412345-284.txt"
}
```

The `diskStorage` callback-based API follows the Node.js error-first convention (Section 3): the first argument to `cb` is an error (or `null` on success), the second is the value. The `path.extname()` function (Section 5) extracts the file extension from the original filename.

Stage 3 Multiple File Uploads

Multer supports multiple files on the same field with `upload.array()` :

```
app.js (multi-upload route)
```

```
// Accept up to 5 files on the "photos" field
app.post('/upload-many', upload.array('photos', 5), function (req, res) {
  console.log('Files received:', req.files.length);
  const summary = req.files.map(function (file) {
    return { name: file.filename, size: file.size };
  });
  res.json({ message: 'Files uploaded', files: summary });
});
```

```
curl -X POST http://localhost:3000/upload-many -F "[email protected]" -F "[email protected]"
```

Output:

```
// Terminal
Files received: 2

// Response
{
  "message": "Files uploaded",
  "files": [
    { "name": "1750070415000-412.txt", "size": 28 },
    { "name": "1750070415001-913.txt", "size": 28 }
  ]
}
```

With `upload.array()`, uploaded files are available on `req.files` (plural) as an array. The second argument (`5`) limits the maximum number of files accepted. Multer responds with a `MulterError` if the client sends more than the limit.

Filtering File Types

Multer's `fileFilter` option lets you reject files based on their MIME type:

app.js (with file filter)

```
const uploadFiltered = multer({
  storage: storage,
  fileFilter: function (req, file, cb) {
    const allowed = ['image/jpeg', 'image/png', 'image/gif'];
    if (allowed.includes(file.mimetype)) {
      cb(null, true); // Accept the file
    } else {
```

```
        cb(new Error('Only JPEG, PNG, and GIF images are allowed'), false);
    }
},
limits: {
    fileSize: 5 * 1024 * 1024 // 5 MB maximum
}
});

app.post('/upload-image', uploadFiltered.single('image'), function (req, res) {
    res.json({ message: 'Image uploaded', filename: req.file.filename });
});
```

```
curl -X POST http://localhost:3000/upload-image -F "[email protected]"
```

Output:

Error: Only JPEG, PNG, and GIF images are allowed

The `fileFilter` callback receives each file before it is saved. Calling `cb(null, true)` accepts the file; `cb(error, false)` rejects it. The `limits` option restricts file size — multer throws a `MulterError` with code `LIMIT_FILE_SIZE` if a file exceeds the maximum.

Important: MIME types sent by the client can be spoofed. The `fileFilter` provides a first line of defence, but for production systems you should also inspect the file's actual content (magic bytes) after upload. Relying solely on the client-provided MIME type is not secure.

Combining Multiple Input Sources

Real-world routes frequently use data from multiple sources in a single handler. Here is an example that combines route parameters, query strings, request body, and headers:

```
app.js
```

```
const express = require('express');
const app = express();

app.use(express.json());

app.put('/api/users/:id', function (req, res) {
  const userId = req.params.id;
  const fields = req.query.fields;
  const updateData = req.body;
  const authToken = req.get('Authorization');

  console.log('--- Request Breakdown ---');
  console.log('Param (id):', userId);
  console.log('Query (fields):', fields);
  console.log('Body:', updateData);
  console.log('Header (auth):', authToken);

  res.json({
    updated: userId,
    fields: fields || 'all',
    data: updateData,
    authenticated: !!authToken
  });
});

app.listen(3000, function () {
  console.log('Server running on http://localhost:3000');
});
```

```
curl -X PUT "http://localhost:3000/api/users/42?fields=name,email" -H "Content-Type: application/json" -H "Authorization: Bearer tok_abc" -d "{\"name\": \"Alice Updated\"}"
```

Output:

```
// Terminal
--- Request Breakdown ---
Param (id): 42
Query (fields): name,email
Body: { name: 'Alice Updated' }
Header (auth): Bearer tok_abc

// Response
{
  "updated": "42",
  "fields": "name,email",
```

```
"data": { "name": "Alice Updated" },  
"authenticated": true  
}
```

This pattern is typical of RESTful APIs: the URL param identifies *which* resource to update, the query string specifies *which fields* to include in the response, the body carries the *new data*, and the header provides *authentication*. Each source has a clear, distinct purpose.

Other Useful `req` Properties

Cross-reference: Common request properties such as `req.method`, `req.path`, `req.url`, `req.hostname`, `req.ip`, `req.protocol`, and `req.headers` were demonstrated in **subtopic 07-01 (Express Introduction), Section 7**. Two additional properties worth noting:

`req.fresh` is a boolean that indicates whether the client's cached version of the response is still valid (based on `ETag` / `If-None-Match` headers). `req.xhr` is `true` when the request was sent via `XMLHttpRequest` — useful for distinguishing AJAX calls from regular browser navigation.

Summary

Concept	Key Point
<code>req.params</code>	Named URL path segments defined with <code>:name</code> in route patterns; always strings
<code>req.query</code>	Key-value pairs from the query string (<code>?key=value</code>); parsed automatically; always strings
<code>req.body</code>	

Concept	Key Point
	Request payload from POST/PUT/PATCH; requires <code>express.json()</code> or <code>express.urlencoded()</code> middleware
<code>express.json()</code>	Parses <code>application/json</code> bodies into <code>req.body</code>
<code>express.urlencoded()</code>	Parses <code>application/x-www-form-urlencoded</code> bodies into <code>req.body</code> ; <code>extended: true</code> supports nested objects
<code>req.headers</code>	All HTTP headers as a lowercase-keyed object; use <code>req.get()</code> for case-insensitive lookup
<code>multer</code>	Third-party middleware for handling <code>multipart/form-data</code> file uploads
<code>upload.single(field)</code>	Accepts one file; metadata on <code>req.file</code>
<code>upload.array(field, max)</code>	Accepts multiple files on one field; metadata on <code>req.files</code>
<code>diskStorage</code>	Custom storage engine for controlling destination directory and filename
<code>fileFilter</code>	Callback to accept or reject files by MIME type before saving
Body size limits	Both <code>express.json()</code> and <code>express.urlencoded()</code> accept a <code>limit</code> option (default <code>100kb</code>)
Params vs query convention	Params identify resources (<code>/users/42</code>); query strings filter/sort/paginate (<code>?page=2</code>)

Interview Questions

1. What are the three main sources of data on an Express request object, and how do they differ?

Expected answer: `req.params` holds named URL path segments (e.g., `/users/:id`). `req.query` holds key-value pairs from the query string (e.g., `?page=2`). `req.body` holds data sent in the request payload (POST/PUT). Params and query are parsed automatically; the body requires middleware like `express.json()`.

2. Why is `req.body` undefined when no body-parsing middleware is registered?

Expected answer: Express does not parse the request body by default. The raw body arrives as a stream of bytes. Without middleware to read and parse that stream (e.g., `express.json()`), Express never populates `req.body`, so it remains `undefined`.

3. What does the `extended: true` option do in `express.urlencoded()`?

Expected answer: When `extended` is `true`, Express uses the `qs` library to parse URL-encoded data, which supports nested objects and arrays (e.g., `sort[field]=name`). When `false`, it uses the built-in `querystring` module, which only supports flat key-value pairs.

4. Route parameters are always strings. How would you safely convert `req.params.id` to a number?

Expected answer: Use `parseInt(req.params.id, 10)` or `Number(req.params.id)`. Then check for `NaN` with `Number.isNaN()` to handle invalid input before using the value in logic or database queries.

5. What is the difference between `req.headers['content-type']` and `req.get('Content-Type')`?

Expected answer: Functionally, both return the same value. `req.headers` stores all headers with lowercase keys, so you must use the exact lowercase form. `req.get()` (or its alias `req.header()`) performs a case-insensitive lookup, so it works regardless of the casing you pass in.

6. Why can't `express.json()` or `express.urlencoded()` handle file uploads?

Expected answer: File uploads use the `multipart/form-data` encoding, which has a completely different structure from JSON or URL-encoded data. It uses boundary delimiters to separate individual fields and binary file content. A specialised parser like `multer` is needed to read and decode this format.

7. Explain the difference between `upload.single()`, `upload.array()`, and `upload.fields()` in `multer`.

Expected answer: `upload.single('fieldName')` accepts exactly one file on the named field and populates `req.file`. `upload.array('fieldName', max)` accepts multiple files on one field and populates `req.files` as an array. `upload.fields([...])` accepts files on multiple named fields, with each field having its own max count, and populates `req.files` as an object keyed by field name.

8. A client sends a `PUT /api/products/99?notify=true` request with a JSON body. Where does each piece of data live on the `req` object?

Expected answer: `req.params.id` (assuming the route is `/api/products/:id`) contains `"99"`. `req.query.notify` contains `"true"`. `req.body` contains the parsed JSON object from the request payload. All three are populated from different parts of the HTTP request.

Practical Assignment: Build a Product API with File Upload

Objective: Create an Express application that accepts data from all four input sources (params, query, body, files) across multiple routes, demonstrating thorough request handling.

1. Create a new project directory called `product-api` and initialise it:

```
mkdir product-api && cd product-api
```

```
npm init -y
```

```
npm install express multer
```

```
mkdir uploads
```

2. Create `app.js` with the following middleware (in order):

1. `express.json()`

2. `express.urlencoded({ extended: true })`

3. A custom request logger that logs the timestamp, method, URL, and content type header for every request.

3. Implement a simple in-memory array to store products (`let products = []`) and define these routes:

1. `GET /api/products` — returns all products. Support `?sort=name` and `?limit=N` query parameters to sort alphabetically by name and limit the number of results.
2. `GET /api/products/:id` — returns a single product by its `id` . Respond with `404` if not found.
3. `POST /api/products` — creates a new product from `req.body` . Expect `{ "name": "...", "price": ... }` . Auto-generate an `id` (use the array length + 1). Respond with `201` .
4. `POST /api/products/:id/image` — uploads a product image using multer. Accept a single file on the field `image` . Store the filename on the matching product object. Filter to only allow JPEG and PNG files. Respond with `404` if the product does not exist.

4. Add a 404 catch-all and an error-handling middleware at the end.

5. Test every route with the following commands and verify the outputs match:

Expected output:

```
curl -X POST http://localhost:3000/api/products -H "Content-Type: application/json" -d '{"name": "Widget", "price": 9.99}'
```

Output:

```
{"message": "Product created", "product": {"id": 1, "name": "Widget", "price": 9.99, "image": null}}
```

```
curl -X POST http://localhost:3000/api/products -H "Content-Type: application/json" -d '{"name": "Gadget", "price": 24.50}'
```

Output:

```
{"message": "Product created", "product": {"id": 2, "name": "Gadget", "price": 24.5, "image": null}}
```

```
curl "http://localhost:3000/api/products?sort=name&limit=1"
```

Output:

```
[{"id":2,"name":"Gadget","price":24.5,"image":null}]
```

```
curl http://localhost:3000/api/products/1
```

Output:

```
{"id":1,"name":"Widget","price":9.99,"image":null}
```

```
curl http://localhost:3000/api/products/999
```

Output:

```
{"error":{"message":"Product
```

